Sample Report

Author McWriterson

Winter 2024/2025

English translation of the Polish original. Historical results and formulas retained.

Contents

1 Task 1 **1**
1.1 Task: . . . 1
1.2 Questions: . . . 2
1.3 Results: . . . 2
1.4 Conclusions: . . . 2

2 Task 2 **2**
2.1 Task: . . . 2
2.2 Questions: . . . 3
2.3 Results: . . . 3
2.4 Conclusions: . . . 3

3 Task 3 **3**
3.1 Task: . . . 3
3.2 Questions: . . . 3
3.3 Results: . . . 3
3.4 Conclusions: . . . 3

4 Task 4 **4**
4.1 Iterative methods . . . 4
4.2 Task . . . 4
4.2.1 Jacobi method . . . 4
4.2.2 Gauss–Seidel method: . . . 4
4.2.3 SOR method: . . . 4
4.3 Results and conclusions: . . . 4
4.3.1 Parameter k . . . 5
4.3.2 Parameter c . . . 5
4.4 Suggestions . . . 5

1 Task 1

1.1 Task:

Given: x, e: column vectors in $\mathbb{R}^n$.

$$k = \frac{\|x\|}{\|e\|} = \sqrt{\frac{x^T x}{e^T e}}, \quad v = x - ke, \quad u = x + ke.$$

Write a function

```
function [P, Q] = haus(x, e)
```

returning matrices P and Q given by:

$$P = I - 2\frac{vv^T}{v^T v}, \quad Q = I - 2\frac{uu^T}{u^T u}.$$

Function call:

```
x = [1; 0; 0; 0];
e = [1/2; -1/2; -1/2; -1/2];
[P, Q] = haus(x, e)
```

< from the slides >

1.2 Questions:

1.3 Results:

n	k	Error 1	Error 2	Error 3	Error 4
3	1.0892	$2.2204e-16$	$2.2204e-16$	$2.2204e-16$	$2.2204e-16$
3000	0.9829	$1.7764e-15$	$1.9984e-15$	$3.2198e-15$	$2.9976e-15$
3000; e almost equal to x	1.000	$1.9984e-15$	$1.8874e-15$	$2.6645e-15$	$3.8858e-15$
3000; x, e orthogonal:	0.9939	$1.9984e-15$	$1.7764e-15$	$4.2188e-15$	$3.5527e-15$

1.4 Conclusions:

2 Task 2

2.1 Task:

Given: $B := A$, a nonsingular matrix.

QR algorithm:

for $m := 1$ **to** n **do**

$t := \sum_{s=m}^{n} (B[s,m])^2$;

if $B[m,m] < 0$ **then** $k := \sqrt{t}$ **else** $k := -\sqrt{t}$;

$d[m] := k$;

$f[m] := \frac{1}{k \cdot B[m,m] - t}$;

$B[m,m] := B[m,m] - k$;

for $i := m + 1$ **to** n **do**

$y := f[m] \sum_{s=m}^{n} B[s,m]B[s,i]$;

for $j := m$ **to** n **do** $B[j,i] := B[j,i] + B[j,m] \cdot y$;

Result:

$$R = \begin{bmatrix} d[1] & B[1,2] & \cdots & B[1,n] & \\ 0 & d[2] & B[2,3] & \cdots & B[2,n] \\ \vdots & \ddots & \ddots & \cdots & \\ 0 & \cdots & d[n-1] & B[n-1,n] & \\ 0 & \cdots & 0 & d[n] & \end{bmatrix}, \quad Q = AR^{-1}.$$

Write a function:

```
function [Q, R] = QR(A)
```

returning Q and R, then find QR decompositions of randomly sampled matrices.

2.2 Questions:

2.3 Results:

n	error-1	error-2	error-3	error-4
3	$8.8818e-16$	$5.5511e-17$	$4.4409e-16$	$3.3307e-16$
500	$5.4368e-13$	$3.5416e-14$	$1.1102e-15$	$7.9381e-15$
100	$3.9968e-15$	$5.6843e-14$	$1.7764e-15$	$3.1264e-13$

2.4 Conclusions:

3 Task 3

3.1 Task:

For the matrix

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix}$$

The matrix 2-norm of A is the maximum over the roots of the characteristic polynomial of A^TA, that is:

$$w(x) := \det(A^TA - xI), \quad C := \{x : w(x) = 0\}, \quad \|A\|_2 = \max_{x \in C} \sqrt{x}.$$

Write a function

```
ret = norm2(A)
```

returning the matrix 2-norm of A.

3.2 Questions:

3.3 Results:

3.4 Conclusions:

Comparing our *norm*2 function with Octave's built-in *norm*, we obtain discrepancies due to numerical errors. Our function's errors mainly come from computing characteristic-polynomial roots, an operation particularly sensitive to closely spaced eigenvalues. Complex roots should not occur; however,

for flat matrices the errors may be complex because the polynomial roots have small imaginary parts. Rounding during characteristic-polynomial computation can also produce unwanted complex roots for large matrices. Large diagonal values can lead to incorrect rounding caused by memory overflow at extreme values. To avoid root-computation errors, one can use the power method. It focuses on the largest eigenvalue, which becomes more apparent across iterations, and requires only matrix–vector multiplication, reducing the risk of errors.

4 Task 4

4.1 Iterative methods

4.2 Task

4.2.1 Jacobi method

Given: A: an $n \times n$ matrix; b, x_0: column vectors in $\mathbb{R}^n$; k: a natural number.
Let $B := D$.
Write a function

```
function retval = Jacobi(A, b, x0, k)
```

returning the vector $x^{(k)}$.

4.2.2 Gauss–Seidel method:

Given: A: an $n \times n$ matrix; b, x_0: column vectors in $\mathbb{R}^n$; k: a natural number.
Let $B := D + L$.
Write a function

```
function retval = GaussSiedel(A, b, x0, k)
```

returning the vector $x^{(k)}$.

4.2.3 SOR method:

Given: A: an $n \times n$ matrix; b, x_0: column vectors in $\mathbb{R}^n$; k: a natural number; $t \in (0, 2)$.
Let $B := \frac{1}{t}D + L$.
Write a function

```
function retval = SOR(A, b, x0, k, t)
```

returning the vector $x^{(k)}$.

4.3 Results and conclusions:

For all calculations we set $x = 0, b = rand(n, 1) * 10$, perform 1000 trials and average their results.

4.3.1 Parameter k

We estimate k, the number of iterations needed to establish convergence. To avoid errors due to a lack of diagonal dominance, set $c = n \cdot 10$. This gives a dominant diagonal and removes errors arising from matrix structure. We can then set $n = 10$, because this matrix size avoids size-related and convergence problems.

Method	1	3	5	10	16
Jacobi	4.0556e-03	6.5079e-06	9.9949e-09	2.7341e-15	1.1819e-17
Gauss-Siedel	3.3702e-03	4.4534e-07	4.7716e-12	2.6600e-17	1.8228e-17
SOR	0.010987	1.3029e-04	1.2422e-06	2.5774e-11	2.3522e-17

Conclusions:

- SOR: effect of a poorly chosen relaxation factor:
  Although SOR should theoretically converge fastest, our calculations do not confirm this. A fixed, poorly chosen relaxation factor was used, reducing its efficiency. A poor relaxation factor can therefore substantially reduce the method's effectiveness.
- Comparison of Jacobi and Gauss–Seidel:
  The results show that Gauss–Seidel converges faster from the first iteration because new values are immediately reused within it. Jacobi uses only preceding-iteration values, slowing convergence.

4.3.2 Parameter c

We estimate c, the value added to the diagonal in $A + c \cdot \text{eye}(n)$. Set $k = 100$ to ensure convergence.

- Case $c = n/2$

Method	3	10	100
Jacobi	$1.4872e-08$	$2.0980e-03$	$8.8618e-03$
Gauss-Siedel	$5.8098e-16$	$3.9518e-16$	$17138e-16$
SOR	$5.9233e-16$	$3.9828e-16$	$17124e-16$

- Case $c = 10 * n$

Method	3	10	100
Jacobi	$3.9344e-17$	$2.5617e-17$	$1.0956e-17$
Gauss-Siedel	$4.0613e-17$	$2.5840e-17$	$1.0976e-17$
SOR	$3.7866e-17$	$2.5573e-17$	$1.0902e-17$

Conclusions:
The approximation of c suggested by the mean $E[\sum_i^n X_i]$, where $X_1 \sim \text{U}([0,1])$, appears to be a good estimate. The Jacobi errors must arise from the algorithm itself, but I do not understand why they occur for [sentence unfinished in the original].

4.4 Suggestions

Stopping criterion:
Choose p and define a stopping time at step n when the following condition holds:

$$\left| \frac{x_n - x_{n-1}}{x_n} \right| \times 100 < p$$